# Common Engineering Program

# Programming Report

# Title: Mini Project Title

## Students name:

Angelin Irwanto (244876B)

Julius Lee Jia Ler (244926F)

Joshua Thomas Lie (242996J)

# Table of Contents

# Report

## Objective(s)

### What is the purpose of the program

The main purpose of this program is to create a digital, user-friendly shopping experience that allows users to view and browse a menu of groceries and easily pick which item they want and modify them in the shopping cart (add and remove items). The program will then calculate the total cost of the items they have chosen to purchase and print a receipt accordingly.

### What This Program Does

This program has functions that allow users to view a menu of various groceries, choose items based on categories, search for specific items based on the item code and choose multiple quantities of items to buy. They can also view their shopping cart as well as add and remove items from their shopping cart. At checkout, users can state whether they are eligible for discounts and see their total owing in a digital receipt.

# Scope

## Key Specifications and Features

### Product Selection and Cart Management

- **User Interaction:** The program allows users to browse through a list of product categories, select products, and add them to their shopping cart. This interaction supports dynamic modifications, where users can add or remove products as needed.
- **Visual Feedback:** Users receive continuous visual feedback through the display of the current contents of their shopping cart, including item names, quantities, and cumulative pricing.

### Menu Display:

Organised Presentation

- **Category-Based Display:** Products are categorised (e.g., Drinks, Snacks, Household) to enhance user navigation and selection efficiency.
- **Clear Information Layout:** Each item within a category is displayed with its name, unique code, and price, facilitating easy recognition and selection.

### Category Selection

Streamlined User Flow

- **Prompt-Driven Selection:** Users are prompted to choose a category, which then leads to the display of relevant items within that category. This structured flow ensures a user-friendly experience that minimises browsing complexity.

## Shopping Cart

Flexible and Detailed Cart Management

- **Interactive Cart Updates:** Users can add items in varying quantities and have the flexibility to modify these entries (add more or remove items) before finalising their purchase.
- **Cart Summary:** The cart displays a detailed summary, including the total number of items, individual item details, and total price, enhancing transparency in shopping decisions.

## Discounts

Inclusive Discount System

- **Eligibility-Based Discounts:** Discounts are available for seniors, members, and NS Men, encouraging a broader user base. The system prompts users to declare their eligibility and automatically applies the appropriate discounts, simplifying the checkout process.

## Checkout and Payment

Efficient and Transparent Billing

- **Comprehensive Receipts:** Post-checkout, the program generates a detailed receipt that includes a breakdown of all purchases, quantities, the total price before discounts, applicable GST, and the grand total.
- **Accurate Calculations:** Ensures accurate computation of totals, GST, and discounts, providing users with a reliable summary of their transaction.

## Major Deliverables

### Robust System Components

- **User Interface:** The program leverages a text-based menu system, allowing for straightforward navigation and selection within a console or terminal environment.
- **Shopping Cart Functionality:** Empowers users with full control over their shopping cart contents, including the ability to view, add, and remove items dynamically.
- **Payment Process:** Seamlessly integrates discount application and GST calculations, ensuring users are billed correctly based on their selections and eligibility.
- **Error Handling:** Implements robust input validation to prevent user errors during category selection, item addition, and during the checkout process, ensuring a smooth user experience.

# Implementation

## Flowchart or Pseudocode

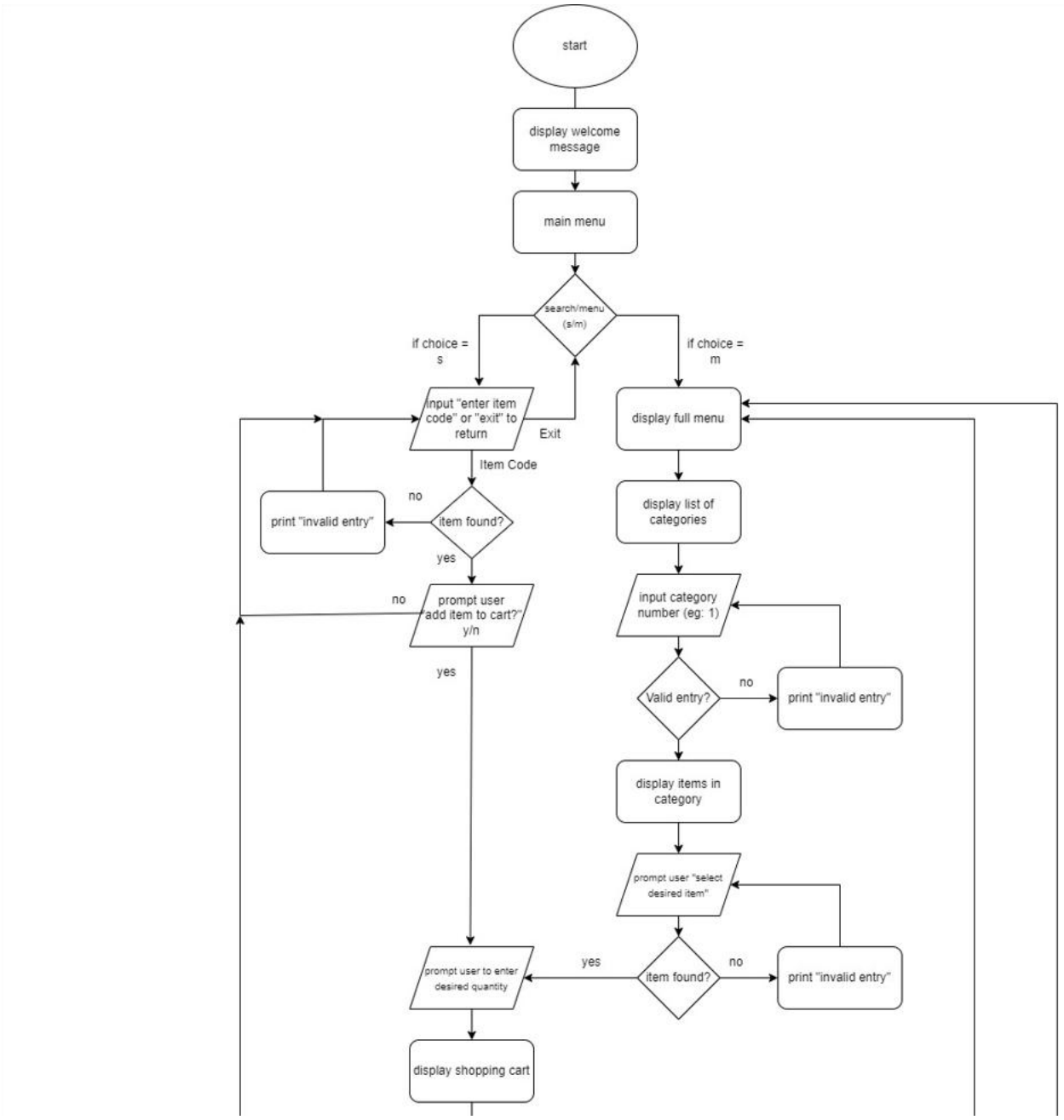



*Figure 1: Flowchart part 1*

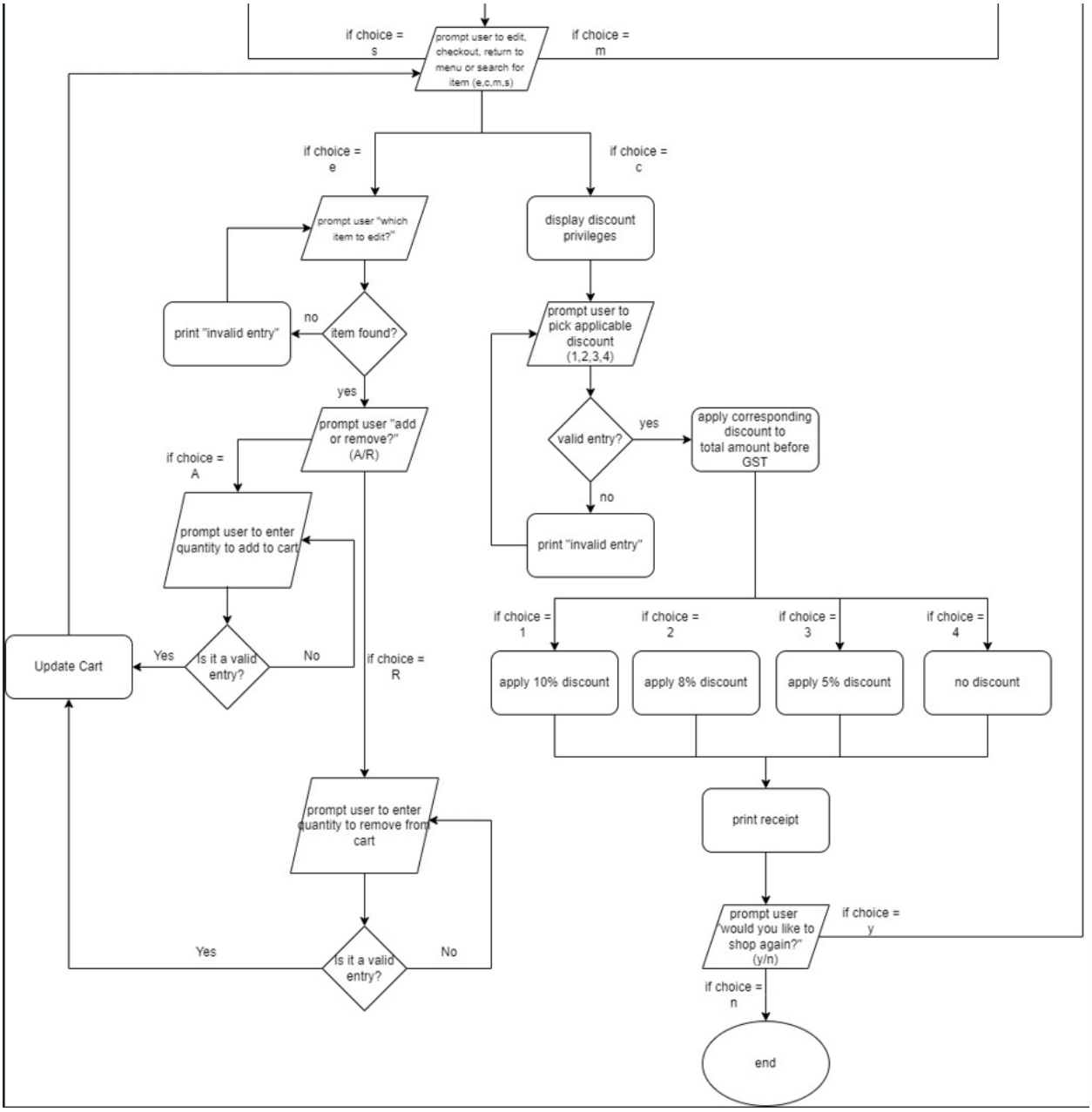
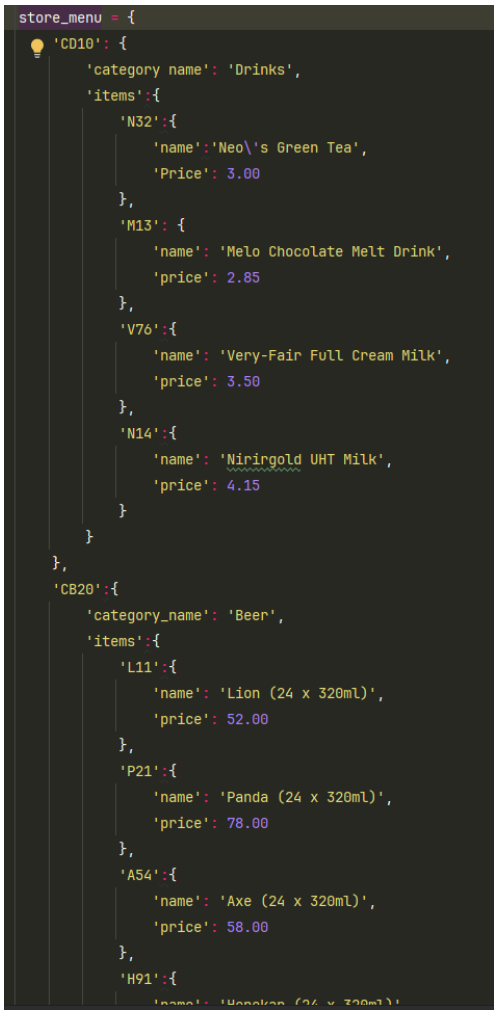



*Figure 2: Flowchart part 2*

## Key technical Issues and Solutions

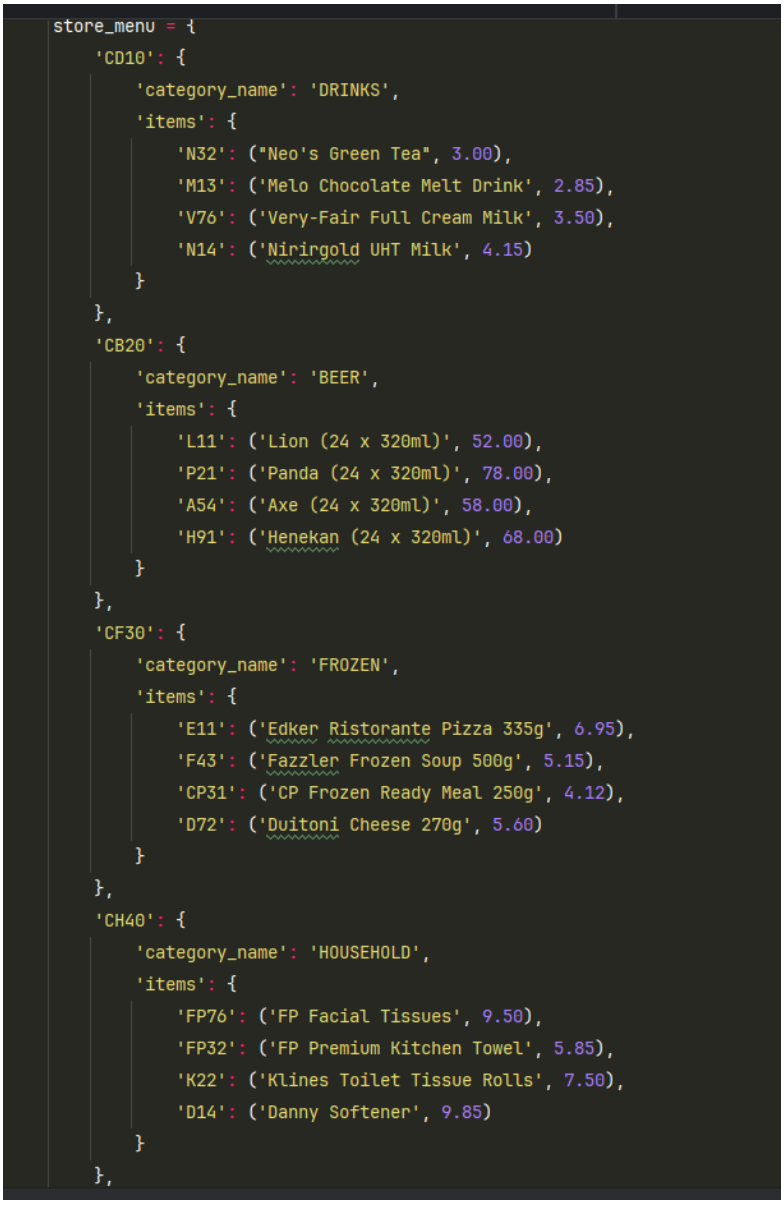
- Organising Product Catalogue Efficiently.
    - Challenge: We needed a way to organise a large number of products across different categories that would be easy to update and quick to access. User Account System and Purchase History.



```
store_menu = {
    'CD10': {
        'category name': 'Drinks',
        'items':{
            'N32':{
                'name':'Neo\'s Green Tea',
                'Price': 3.00
            },
            'M13': {
                'name': 'Melo Chocolate Melt Drink',
                'price': 2.85
            },
            'V76':{
                'name': 'Very-Fair Full Cream Milk',
                'price': 3.50
            },
            'N14':{
                'name': 'Nirirgold UHT Milk',
                'price': 4.15
            }
        }
    },
    'CB20':{
        'category_name': 'Beer',
        'items':{
            'L11':{
                'name': 'Lion (24 x 320ml)',
                'price': 52.00
            },
            'P21':{
                'name': 'Panda (24 x 320ml)',
                'price': 78.00
            },
            'A54':{
                'name': 'Axe (24 x 320ml)',
                'price': 58.00
            },
            'H91':{
```

*Figure 3: Old Dictionary*

- Solution: We chose to use nested dictionaries with tuple to structure our product catalogue. This allowed us to group items by category and quickly look up the product details using unique item codes. Using a tuple also prevents any changes of items within the codes.



```
store_menu = {
    'CD10': {
        'category_name': 'DRINKS',
        'items': {
            'N32': ("Neo's Green Tea", 3.00),
            'M13': ('Melo Chocolate Melt Drink', 2.85),
            'V76': ('Very-Fair Full Cream Milk', 3.50),
            'N14': ('Nirirgold UHT Milk', 4.15)
        }
    },
    'CB20': {
        'category_name': 'BEER',
        'items': {
            'L11': ('Lion (24 x 320ml)', 52.00),
            'P21': ('Panda (24 x 320ml)', 78.00),
            'A54': ('Axe (24 x 320ml)', 58.00),
            'H91': ('Henekan (24 x 320ml)', 68.00)
        }
    },
    'CF30': {
        'category_name': 'FROZEN',
        'items': {
            'E11': ('Edker Ristorante Pizza 335g', 6.95),
            'F43': ('Fazzler Frozen Soup 500g', 5.15),
            'CP31': ('CP Frozen Ready Meal 250g', 4.12),
            'D72': ('Duitoni Cheese 270g', 5.60)
        }
    },
    'CH40': {
        'category_name': 'HOUSEHOLD',
        'items': {
            'FP76': ('FP Facial Tissues', 9.50),
            'FP32': ('FP Premium Kitchen Towel', 5.85),
            'K22': ('Klines Toilet Tissue Rolls', 7.50),
            'D14': ('Danny Softener', 9.85)
        }
    },
```

*Figure 4: New Dictionary System*

- Ensuring Robust User Input Handling
    - Challenge: Users might enter invalid inputs, such as non-existent item codes or negative quantities, which could crash the program.
    - Solution: We implemented a series of input validation checks using while loops and try-except blocks. This would guide users to provide correct inputs, enhancing the overall user experience and preventing program crashes.

```
    try:
        print()
        item_index = int(input("Please add desired item to cart (eg:1): "))
        if 1 <= item_index <= len(item_codes):
            item_code = item_codes[item_index - 1]
            break
        else:
            print()
            print('Invalid entry, please try again.')
    except ValueError:
        print()
        print('Invalid entry, please enter a valid number.')

while True:
    try:
        print()
        quantity = int(input('Please add the desired quantity to cart (eg: 1): '))
        if quantity <= 0:
            print()
            print("Invalid quantity, please enter a valid positive number.")
        else:
            break
    except ValueError:
        print()
        print('Invalid quantity, please enter a valid number.')
```

*Figure 5: Example of Error Handling*

- Streamlined Shopping Cart Management
    - Challenge: Keeping track of items in the cart, including adding new items, updating quantities, and removing items efficiently.
    - Solution: We developed dedicated functions like add_items, remove_items, and display_cart to manage the shopping cart. These functions work with a central shopping_cart dictionary, ensuring that all cart operations are consistent and reflected immediately in the user interface.

```
3 usages
def add_item(item_code, item_name, quantity, item_price):
    """
    Adds a certain quantity of items to the shopping cart
    Assumption: All variables are valid
    :param item_code: code of the item to add
    :param item_name: name of the item to add
    :param quantity: quantity of the item to add
    :param item_price: price of the item to add
    :return: None, perform operation in place (shopping_cart)
    """
    if item_code in shopping_cart:
        shopping_cart[item_code][1] += quantity
    else:
        shopping_cart[item_code] = [item_name, quantity, item_price]
```

*Figure 6: add_item function*

```
def remove_item(item_code, remove_quantity):
    if item_code in shopping_cart:
        shopping_cart[item_code][1] -= remove_quantity
        if shopping_cart[item_code][1] <= 0:
            del shopping_cart[item_code]
    else:
        print("Item is not in shopping cart")
```

*Figure 7: remove_item function*

## Special Features

### Special Feature: Search Function

In our code, we decided to implement a search feature as our special feature.

```python
def search_item():
    while True:  # Encapsulate the search code in a loop to keep prompting until a valid item is found or until the user wishes
        print()
        search_code = input("Enter item code to search (eg: N32) or type 'exit' to return: ").upper()
        if search_code == 'EXIT':
            break
        item_found = False

        for category_data in store_menu.values():
            if search_code in category_data['items']:
                item_found = True
                item_name = category_data['items'][search_code][0]
                item_price = category_data['items'][search_code][1]
                print(f"\nFound: {item_name} - ${item_price:.2f}\n")

                while True:
                    add_to_cart = input("Would you like to add this item to the cart? (Y/N): ").upper()
                    if add_to_cart == "Y":
                        while True:
                            try:
                                quantity = int(input("Enter the quantity to add to the cart: "))
                                if quantity > 0:
                                    add_item(search_code, item_name, quantity, item_price)
                                    print(f"Added {quantity} of {item_name} to the cart.")
                                    return  # Exit after adding item
                                else:
                                    print()
                                    print("Please enter a positive quantity.")
                            except ValueError:
                                print()
                                print("Invalid quantity. Please enter a number.")
                            break # Break the inner loop after processing the valid input
                    elif add_to_cart == "N":
                        return  # Exit without adding if the user chooses not to add the item
                    else:
                        print()
                        print("Invalid entry, please enter 'Y' for yes or 'N' for no.")

        if not item_found:
            print("Item not found. Please try again.")
```

*Figure 8: search_item function*

This feature prompts users to immediately input the item they want using the item's code that was listed in the menu instead of needing to go through the process of selecting the category. This allows those who have made up their mind to have more convenience when adding items to cart.

## Why You Designed the Program This Way

### Enhanced Accessibility and User Experience

**Immediate Feedback:** Upon entering a search query, users receive immediate feedback. If the item exists, they see the item details and are given the option to add it directly to their cart. If no matches are found, the system informs the user, allowing them to adjust their search terms or exit the search.

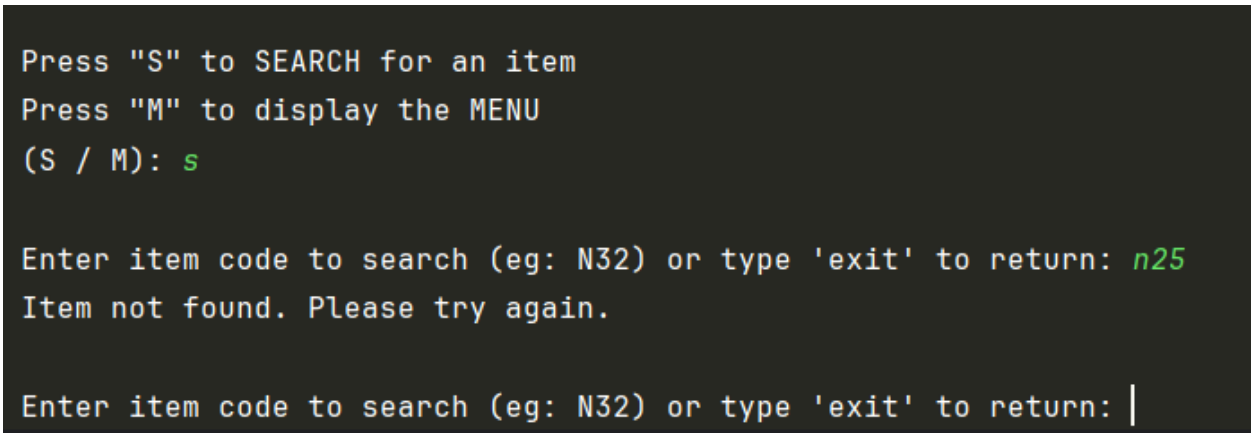



*Figure 9: Ease of search function usage*

**User-Centric Design:** The design of the search functionality is grounded in the principle of user convenience. Recognizing that users may visit the store with specific products in mind, this feature minimises the browsing required, making the shopping process quicker and more user-friendly.

### Integration into the Shopping Experience

**Seamless Integration:** The search function is seamlessly integrated into the main shopping flow. Users can invoke this feature at any point during their shopping session by selecting the search option from the main menu.

```
Enter item code to search (eg: N32) or type 'exit' to return: exit


Press "S" to SEARCH for an item
Press "M" to display the MENU
(S / M):
```

*Figure 10: Ease of navigation through search function*

**Flexibility in Shopping Navigation:** By allowing users to search at any stage of their shopping process, the system accommodates diverse shopping strategies—whether users prefer to browse through categories or know exactly what they want.

# Conclusion

## Accomplishment / Outcomes

The outcome of the program was to allow users to successfully view and select desired items in desired quantities before proceeding to checkout and purchasing the item(s).

## Future Enhancement

1. Enhanced User Experience:
    - Implementation of user accounts would allow for personalised shopping experiences. This system could include features such as saved shopping carts, purchase history tracking, and personalised product recommendations based on past buying behaviour. Users could easily reorder favourite items and pick up shopping where they left off, significantly improving convenience and user satisfaction.

2. Data-Driven Business Insights:

- A user account system with purchase history would provide valuable data for business analytics. This data could be used to inform inventory management, tailor marketing strategies, and implement targeted promotions. Additionally, it would enable the creation of loyalty programs, encouraging repeat customers and fostering long-term customer relationships. From a customer service perspective, having access to purchase histories would facilitate quicker resolution of order-related inquiries.

These features represent a significant opportunity for future enhancement, potentially transforming the shopping system into a more robust and user-centric platform.

## Appendices

Appendix A (view menu)

| Category | Item | Code | Price |
|---|---|---|---|
| DRINKS | Neo's Green Tea | N32 | $3.00 |
| | Melo Chocolate Melt Drink | M13 | $2.85 |
| | Very-Fair Full Cream Milk | V76 | $3.50 |
| | Nirirgold UHT Milk | N14 | $4.15 |
| | | | |
| BEER | Lion (24 x 320ml) | L11 | $52.00 |
| | Panda (24 x 320ml) | P21 | $78.00 |
| | Axe (24 x 320ml) | A54 | $58.00 |
| | Henekan (24 x 320ml) | H91 | $68.00 |
| | | | |
| FROZEN | Edker Ristorante Pizza 335g | E11 | $6.95 |
| | Fazzler Frozen Soup 500g | F43 | $5.15 |
| | CP Frozen Ready Meal 250g | CP31 | $4.12 |
| | Duitoni Cheese 270g | D72 | $5.60 |
| | | | |
| HOUSEHOLD | FP Facial Tissues | FP76 | $9.50 |
| | FP Premium Kitchen Towel | FP32 | $5.85 |
| | Klines Toilet Tissue Rolls | K22 | $7.50 |
| | Danny Softener | D14 | $9.85 |
| | | | |
| SNACKS | Singshort Seaweed | SS93 | $3.10 |
| | Mei Crab Cracker | MC14 | $2.05 |
| | Reo Pokemon Cookie | R35 | $4.80 |
| | Huat Seng Crackers | HS11 | $3.55 |
| | | | |

Appendix B (select category)

```
CATEGORIES
----------------------------------------
1. DRINKS
2. BEER
3. FROZEN
4. HOUSEHOLD
5. SNACKS

Please enter desired category number (eg: 1): 5

SNACKS
----------------------------------------
1. SS93: Singshort Seaweed [$3.10]
2. MC14: Mei Crab Cracker [$2.05]
3. R35: Reo Pokemon Cookie [$4.80]
4. HS11: Huat Seng Crackers [$3.55]

Please add desired item to cart (eg:1): 1

Please add the desired quantity to cart (eg: 1): 12
```

Appendix C (Special Feature - Search Function)

```
Press "S" to SEARCH for an item
(E / C / M / S): s

Enter item code to search (eg: N32) or type 'exit' to return:
```

Appendix D (View Shopping Cart)

```
                              SHOPPING CART
-------------------------------------------------------------------------------
ITEM CODE  | ITEM NAME                                | QUANTITY   | TOTAL PRICE
SS93         Singshort Seaweed                        10           $31.00
N32          Neo's Green Tea                          14           $42.00
H91          Henekan (24 x 320ml)                     14           $952.00
D14          Danny Softener                           22           $216.70
F43          Fazzler Frozen Soup 500g                 14           $72.10
```

Appendix E (Removing Items From Shopping Cart)

```
SS93         Singshort Seaweed                          10           $31.00
N32          Neo's Green Tea                            14           $42.00
H91          Henekan (24 x 320ml)                       14           $952.00
D14          Danny Softener                             22           $216.70
F43          Fazzler Frozen Soup 500g                   14           $72.10


Press "E" to edit items in cart
Press "C" to checkout
Press "M" to return to shop menu
Press "S" to search for an item
(E / C / M / S): e


Which item would you like to edit? (eg: N32): ss93


Would you like to add or remove items? (A / R): r


Please enter the amount you would like to remove: 5

                              SHOPPING CART
--------------------------------------------------------------------------------
ITEM CODE  | ITEM NAME                                | QUANTITY   | TOTAL PRICE
SS93         Singshort Seaweed                          5            $15.50
N32          Neo's Green Tea                            14           $42.00
H91          Henekan (24 x 320ml)                       14           $952.00
D14          Danny Softener                             22           $216.70
F43          Fazzler Frozen Soup 500g                   14           $72.10
```

Appendix F (Checkout)

```
Press "E" to edit items in cart
Press "C" to checkout
Press "M" to return to shop menu
Press "S" to search for an item
(E / C / M / S): c


DISCOUNT PRIVILEGES
---------------------------------------
1. Seniors (10%)
2. Members (8%)
3. NS Men (5%)
4. No

Are you eligible for any of these discount privileges? (eg: 1): 4


                                RECEIPT
-------------------------------------------------------------------------------
ITEM CODE  | ITEM NAME                                | QUANTITY   | TOTAL PRICE
SS93         Singshort Seaweed                          5            $15.50
N32          Neo's Green Tea                            14           $42.00
H91          Henekan (24 x 320ml)                       14           $952.00
D14          Danny Softener                             22           $216.70
F43          Fazzler Frozen Soup 500g                   14           $72.10

Total before GST: $1298.30
Discount: $0.00
GST: $116.85
Final Price: $1415.15
Thank you for shopping with us! See you again :))
```